

AI Engineer (Demo Task)

Submission Link - <https://forms.gle/8bnrg78Ki4E25RAk8>

Executive Summary

GraphOne / FrontierAtlas is engineering the premier global Intelligence Graph for the artificial intelligence and venture ecosystem. Our infrastructure facilitates the continuous ingestion, normalization, and enrichment of multi-dimensional datasets encompassing startups, founders, products, AI jobs, research papers, and real-time news signals from thousands of sources worldwide.

This advanced assessment evaluates your technical proficiency in architecting resilient, production-grade data pipelines. We face unique challenges at the intersection of traditional web scraping and LLM-based data extraction. This assignment simulates the actual work performed by our Data Intelligence team.

Core Objectives & Assessment Scope

You are tasked with building a scalable, fault-tolerant ingestion pipeline. We prioritize architectural sophistication, edge-case handling, and raw data fidelity.

Key Evaluation Areas

- **Massive Bulk Extraction:** Architecting a one-time scrape capable of handling hundreds of thousands (lakhs) of records without bottlenecks.
- **Resilient LLM Integration:** Handling rate limits (429s) and context window overflows (413s).
- **Precision Extraction:** Extracting rigorous timestamps for data freshness and tracking dynamic metrics (e.g., GitHub stars).
- **Anti-Bot Navigation:** Bypassing protections like Cloudflare and Datadome.
- **Entity Resolution:** Canonicalizing organizations and products dynamically.

- **Production Readiness:** Concurrency, robust retries, and clean logging.

Phase I: Massive One-Time Data Acquisition

Your system must demonstrate the ability to perform a massive, one-time bulk extraction of entity data. While the end goal for our team is to acquire "lakhs" (hundreds of thousands) of records, you are not expected to scrape 500k records for this trial.

Requirements

- **Scraper Design:** Develop a highly concurrent scraper capable of rapidly acquiring structured data from large directories.
- **Research Papers Vertical:** Extract AI research papers from sources like [Arxiv](#) and [Papers with Code](#). Correlate papers with associated GitHub repositories and extract dynamic metrics like current GitHub stars.
 - Example Task: <https://paperswithcode.co/paper/98456>
- **Scalability:** The architecture must theoretically scale to 500,000+ records without requiring code changes, only infrastructure scaling.
- **Target Output:** Extract a minimum of:
 - 1,000 unique startup records
 - 1,000 unique product records
 - 1,000 unique research papers (with GitHub metrics)

Phase II: High-Fidelity Signal Ingestion

Monitor a set of 5 distinct AI news sources and 5 AI job boards.

The Freshness Challenge

We require extreme freshness. All ingested news and jobs must be guaranteed to have been published within the last 24 hours.

- **Full-Text Content:** Develop an automated crawler that extracts full-text content.

- **Date Normalization:** Implement custom logic to extract and normalize publication dates, handling relative dates (e.g., "2 hours ago") and missing meta tags.
- **Intelligent Heuristics:** If a source lacks a strict date, implement an intelligent heuristic to determine if the content is new since the last run.

Phase III: Multi-Tier LLM Extraction Engine

Raw HTML/Text must be structured into our canonical JSON schema using Large Language Models.

Requirements

- **Fallback Chain:** Implement a multi-tier fallback (e.g., Gemini Flash → Groq Llama 3 → DeepSeek).
- **Intelligent Chunking:** Implement a truncation strategy to ensure payloads never trigger 413 Payload Too Large errors while retaining semantically dense content.
- **Rate Limit Handling:** Handle 429 Too Many Requests gracefully with exponential backoff and jitter.

Phase IV: Deterministic Entity Resolution

Develop a deduplication/resolution engine that canonicalizes Startup and Product names. For example, "OpenAI", "OpenAI, Inc.", and "Open AI" must resolve to the canonical "OpenAI".

Requirements

- **Mapping:** Map extracted entities against a seed list of known canonical entities (you may mock a small database of 50 known AI startups).

Phase V: Anti-Bot & Scale Thinking

Demonstrate or comprehensively document your strategy for scraping Cloudflare-protected or heavily JavaScript-rendered domains without triggering captchas.

Requirements

- **Asynchronous Operation:** The crawler must operate asynchronously (e.g., using `asyncio + aiohttp` or `Playwright Async`).
- **High-Value Sources:** Showcase how you handle high-value intelligence sources that aggressively block automated requests.

Phase VI: Architecture & Production Design

Provide a concise technical architecture document (Max 3 pages) addressing:

1. **Scale Strategy:** How would you collect 500,000 startups, products, and research papers without manual intervention?
2. **Handling 413s & 429s:** Your exact strategy for managing context windows and rate limits across thousands of concurrent extractions.
3. **Freshness Tracking:** How your architecture ensures we never process the same article/job twice across distributed crawler nodes.
4. **Storage Strategy:** Justify your choice of primary database and vector/graph storage for mapping complex relationships.

Expected Schemas

Startup Entity

Field	Type	Description
schemaVersion	String ▾	Versioning for the schema (e.g., "1.0")

Field	Type	Description
recordType	String ▾	Fixed to "STARTUP"
source.name	String ▾	Name of the source site
source.url	String ▾	Original source URL
content.entityName	String ▾	Canonical startup name
content.data.employeeCount	Integer ▾	Number of employees (if available)
collectedAt	Timestamp ▾	ISO-8601 format

Product Entity

Field	Type	Description
schemaVersion	String ▾	Versioning for the schema (e.g., "1.0")
recordType	String ▾	Fixed to "PRODUCT"
source.name	String ▾	Name of the source site
source.url	String ▾	Original source URL
content.startupName	String ▾	Canonical startup name
content.pricingModel	Enum ▾	FREE, FREEMIUM, PAID, ENTERPRISE

Field	Type	Description
collectedAt	Timestamp ▾	ISO-8601 format

Research Paper Entity

Field	Type	Description
schemaVersion	String ▾	Versioning for the schema
recordType	String ▾	Fixed to "RESEARCH_PAPER"
content.title	String ▾	Title of the research paper
content.authors	Array ▾	List of author names
content.paper_url	String ▾	Link to the Arxiv/PDF page
content.github_url	String ▾	Link to the associated code repository (if any)
content.github_stars	Integer ▾	Current number of stars on the GitHub repository
content.published_date	Timestamp ▾	ISO-8601 publication date

Job Entity

Field	Type	Description
schemaVersion	String ▾	Versioning for the schema

Field	Type	Description
recordType	String ▾	Fixed to "JOB"
content.company	String ▾	Canonical company name
content.date	Timestamp ▾	ISO-8601 publication date
content.is_remote	Boolean ▾	Remote eligibility
content.role_family	String ▾	Functional category (e.g., "Engineering")

Deliverables

At the end of the 3-day trial period, you must submit the following:

1. Data Output (Google Sheets)

Provide a public Google Sheet link containing the output of your pipeline across 6 tabs:

- **Startups** (Min. 1,000 rows)
- **Products** (Min. 1,000 rows)
- **Research Papers** (Min. 1,000 rows, including GitHub stars)
- **Jobs** (All 24-hr fresh jobs found)
- **News** (All 24-hr fresh news found)
- **Entity Mapping Log** (Raw vs Canonical names)

Submission Link:

2. Engineering Output (GitHub Repository)

Provide a link to a repository containing:

- **README.md:** Setup instructions and architecture overview.
- **src/:** Source code for the crawler, LLM orchestrator, and entity resolver.
- **architecture.pdf:** Detailed design documentation.

Submission Link - <https://forms.gle/8bnrg78Ki4E25RAk8>

Evaluation Criteria

Category	Weight	Focus Area
LLM Orchestration	25%	Fallback chain robustness and payload chunking
Data Quality	25%	Date parsing accuracy, 24-hour freshness, and GitHub tracking
Scale Thinking	20%	Ability to acquire "lakhs" of records and architect for 500k+
Engineering Rigor	20%	Async design, error handling, and maintainability
Entity Resolution	10%	Precision in mapping messy strings to canonical forms

WARNING: Hallucinated data generated by the LLM will result in immediate disqualification. Every record must trace back to a legitimate, valid source URL.

 **Important Note**

This task is **intentionally challenging**.

We have designed it to create a **steep learning curve** and to identify people who can operate with **high ownership and high agency**.

You are not expected to know everything beforehand.

But you are expected to:

- figure things out independently
- navigate ambiguity
- make decisions without waiting for instructions
- push through technical and product challenges

This is **not a typical internship assignment**.

It reflects the kind of problems you will work on here.

If you're someone who enjoys:

- building from scratch
- solving unclear problems
- taking full ownership

you will find this exciting.

If you prefer structured, step-by-step tasks, this may not be the right fit

Submission Link - <https://forms.gle/8bnrg78Ki4E25RAk8>